Contention-Guided Feedback for Integrated Scheduling of Heterogeneous Robot Arms and Conflict-Free AGV Routing in Flexible Job Shops

Integrated scheduling of processing and transportation resources in flexible job shops is almost always modelled with a constant travel-time matrix, which silently assumes that vehicles never interfere. Once a limited fleet shares a physical corridor network, travel time is no longer input data but an outcome of routing decisions. We study the integrated problem in which each operation may be processed by a subset of robot arms with machine-dependent processing times, while automated guided vehicles must be routed conflict-free over that network. Heterogeneity is what makes the coupling consequential: when the route to a fast arm is congested, a slower but better-connected arm can be the better assignment—a trade-off that existing integrated heuristics cannot act upon, because they either admit a single eligible machine per operation or revisit routing only once, in one direction, after the schedule has been fixed. We propose a closed-loop bilevel framework with two feedback paths. On the evaluation path, the fitness of every individual in every generation is the true makespan obtained after conflict-free routing, including all yielding delays. On the decision path, the routing layer returns *contention certificates* instead of time: the critical chain is attributed into five labelled causes, and corridor-contention labels name the operations to re-assign (a change of place) or to stagger (a change of time), so that neighbourhoods follow the lower layer's evidence rather than random sampling; dispatching is closed likewise, by probing the reservation table. We additionally report a reproducible negative result—pricing this interface with corridor shadow prices is systematically harmful, because contention delay is already internalised in the delayed vehicle's own arrival time and is charged a second time by the price—which argues for passing structural evidence rather than prices in any architecture whose fitness is a complete conflict-free decoding. Experiments use a controlled instance family whose congestion knobs alter network capacity without altering distances. Under an equal wall-clock budget the framework reduces makespan by [TBD]% relative to a two-stage open-loop baseline, while [TBD] of the mechanism gain vanishes once the added congestion is one that no assignment can avoid.

CCS Concepts: • **Computer systems organization** → **Robotic autonomy**.

Additional Key Words and Phrases: Flexible Job Shop Scheduling, Heterogeneous Robot Arms, Conflict-Free AGV Routing, Integrated Scheduling and Routing, Contention-Guided Feedback, Closed-Loop Bilevel Optimization, Memetic Algorithm

ACM Reference Format:

. 2025. Contention-Guided Feedback for Integrated Scheduling of Heterogeneous Robot Arms and Conflict-Free AGV Routing in Flexible Job Shops. 1, 1 (July 2025), 23 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Flexible job shop scheduling with transportation rests on one convenient assumption: the time to move a job between two machines is a constant, read from a matrix indexed by locations. It is the dominant convention in the literature [10] and recent models retain it [2, 6, 7], and it is a reasonable assumption whenever the fleet is small relative to the aisle network, so that vehicles rarely meet. Shop floors, however, are not built that way: a handful of vehicles share a sparse set of aisles, single-lane segments cannot be traversed in opposite directions at the same time, and once a fleet is dense

Author's Contact Information:

Manuscript submitted to ACM

enough its own traffic becomes a first-order source of delay rather than a nuisance [4]. Once that is admitted, travel time stops being data. It becomes the outcome of two decisions—which route a vehicle takes, and who reserved the contested segment first—and the delay it produces is a delay one job inflicts on another rather than a property of the layout.

This matters for scheduling only if the schedule has something to do about it, and that is exactly what machine flexibility provides. When an operation may be processed by any arm in an eligible subset, and the processing time depends on which arm is chosen, the assignment decision selects a destination as well as a duration. Consider an operation that can run on a fast arm at the far end of the shop or on a slower arm next to the load/unload station. Under a constant travel matrix the comparison is arithmetic: shorter processing plus longer travel against the reverse. Under contention it is not, because the aisle leading to the fast arm may be the one every other job is also using, and the waiting incurred there is invisible to the matrix. The slower but better-connected arm can then be the better choice—and the reverse can hold at a different congestion level for the same instance. This trade-off requires both ingredients at once: heterogeneous eligible arms give the scheduler an alternative destination, and an explicit aisle network gives the alternative a different cost. Remove either one and the trade-off disappears.

Existing work either addresses one side of this coupling at a time or couples the two in one direction. Conflict-free multi-robot routing takes contention very seriously—lifelong formulations go as far as reshaping edge costs to steer traffic away from bottlenecks [11]—but tasks, their release times and their endpoints arrive from outside the planner, which is therefore never permitted to question the assignment that created them. Congestion-aware fleet control similarly penalizes congested segments while dispatching a fixed set of transport orders, and measures congestion as local vehicle density [4]. On the scheduling side the combination has already been formalized: the conflict-free-transportation-constrained flexible job shop scheduling problem couples machine-eligible operations having location-dependent processing times with conflict-free vehicle routing, and a logic-based Benders decomposition solves benchmark instances of it to proven optimality [9]. We adopt that problem class rather than claim it. What remains open is the heuristic regime, entered as soon as instances outgrow what exact decomposition reaches, and there the coupling is still one-directional. A two-stage scheme fixes the schedule first and then resolves vehicle conflicts against it [8], so contention discovered in the second stage cannot revise the first. The closest work to ours adds a feedback step but, as its own limitations section states, "operates primarily as a two-stage sequential optimization strategy rather than a fully dynamic, real-time closed-loop system", relying on "a one-time feedback adjustment" [5]. Two consequences follow from that design, and both are structural rather than incidental: the feedback adjusts the time axis without revising any assignment, and it is applied once to a single incumbent rather than to every candidate the search examines. That work is moreover set on job shop benchmarks with a single eligible machine per operation and fixed processing times, where the reassignment trade-off above cannot arise at all.

We close the loop in both senses. Our framework keeps the bilevel structure—an upper layer deciding assignment and sequencing, a lower layer producing conflict-free routes by time-window reservation—but connects the layers along two paths. The *evaluation path* defines the fitness of every candidate, in every generation, as the makespan obtained after the lower layer has actually routed it, so that yielding delays enter the objective the search optimizes instead of being checked after the fact. The *decision path* carries what we call *contention certificates*. Rather than returning a time, the lower layer returns an attribution of the critical chain: the chain is extended from operations to alternating operation–transport links, and each link is assigned one of five causes—`operation`, `machine`, `upstream`, `vehicle` or `corridor`, the last naming the contested segment and the interval in which the yield occurred. Without this distinction a delayed start is merely late, and one cannot tell a slow predecessor from a missing vehicle from a blocked aisle; with it, the certificate names the operations to act on. Two operator families then act on the corridor labels, corresponding to the two ways contention can be relieved: *reassign* moves an operation to an arm reached through a less contested part of the network (a change of place), and *stagger* shifts the blocked operation earlier or its blocker later (a change of time). The distinction from a conventional mutation operator is not the move itself but its argument, which is designated by the lower layer's evidence instead of sampled uniformly. Dispatching is closed the same way: a vehicle is chosen by probing the reservation table with a two-leg trial rather than by estimating arrival from an ideal shortest path, which removes the last open-loop element of the framework.

A natural alternative is to price the interface: let the lower layer export shadow prices for corridor–time slots and let routing and reassignment pay for what they consume. We implemented this and report that it is systematically harmful, monotonically so in the price weight, and equally ineffective when the prices are computed by definitional finite differences instead of a surrogate. The explanation is a double charge. Contention delay is already internalized in the delayed vehicle's own arrival time, propagates through start and completion times into the makespan, and is therefore already counted by the fitness; adding a price makes the vehicle detour or depart later to avoid a cost it has already paid, trading a first-order, certain self-delay for a second-order, uncertain externality. The contrast with lifelong routing is instructive, because there the same idea works: biasing edge costs away from bottlenecks does raise throughput [11], precisely because no outer objective is already charging agents for the delay they inflict. What differs is not the soundness of the idea but whether contention has been counted once already. The inference generalizes: in any architecture whose fitness is a complete conflict-free decoding, penalizing congestion inside the lower layer is likely to hurt, and the interface should be upgraded by passing structural evidence—which changes how the upper layer's neighbourhood is *constructed*—rather than prices, which change the lower layer's objective. This is why the mechanism is contention-*guided* rather than price-coordinated.

Establishing any of this required a stricter experimental protocol than is customary, for two reasons we encountered the hard way. First, the mechanisms differ enormously in cost per evaluation—by up to two orders of magnitude between the cheapest and the most expensive variant—so comparing at equal generation counts credits a mechanism for the extra computation it consumes; under equal wall-clock budget several apparent gains reverse. Second, the seed-to-seed spread of makespan on congested instances exceeds the differences between variants, so a single seed can rank the variants in almost any order. We therefore compare all variants under an equal budget, over at least ten seeds, with paired Wilcoxon tests. A third confound is the instance itself: congestion concentrated at the load/unload exit is traversed by every job regardless of assignment, so it raises every schedule's delay without offering the mechanisms anything to exploit. We generate instances whose knobs vary network capacity while holding distances fixed, record the share of such assignment-irrelevant congestion, and use a matched pair of instances differing only in that share to separate congestion the scheduler can route around from congestion it cannot.

Our contributions are as follows.

(1) **Assignment-relevant congestion, identified and measured.** Working in the established conflict-free-transportation-constrained setting [9], we identify the structural condition under which the coupling carries any decision leverage at all. Congestion at the load/unload exit is traversed by every job whatever the assignment, so it inflates every schedule's delay without rewarding any decision, whereas congestion on the corridors leading to a subset of the arms can be routed around by reassignment. We turn this share into an explicit instance parameter, which turns out to explain why mechanisms that appear inert on a hand-built congested layout become effective on a generated one.

(2) **Contention-guided closed-loop feedback.** We replace time-valued or price-valued interlayer feedback with typed contention certificates, and derive from them two directed operator families and a reservation-aware dispatch rule. Because the lower layer resolves every conflict by waiting within reserved windows, any assignment and sequence decodes to a conflict-free, deadlock-free schedule of finite makespan, so the search never spends effort repairing infeasibility.

(3) **A reproducible negative result with a mechanism explanation.** We show that pricing the interlayer interface degrades performance monotonically, explain it as a double charge on contention already internalized by the fitness, and state the design inference for architectures of this class. We report this in full rather than omitting it, since a negative result with a mechanism is more informative than another incremental heuristic.

(4) **A protocol that separates mechanism from computation.** We evaluate under equal wall-clock budget over at least ten seeds with paired tests, on a controlled instance family in which congestion is varied through capacity alone, and we certify every reported schedule with an independent feasibility validator against a composite lower bound. Under this protocol the closed loop reduces makespan by [TBD]% relative to the two-stage baseline, and [TBD] of the gain disappears when the added congestion is assignment-irrelevant.

The remainder of this paper is organized as follows. Section 2 reviews related work on flexible job shop scheduling with transport, conflict-free multi-robot routing, and integrated formulations. Section 3 presents the problem formulation. Section 4 details the closed-loop bilevel framework, the contention certificates and the operators derived from them. Section 5 reports the experimental protocol and results, including the ablation chain and the negative result on pricing. Section 6 concludes and discusses limitations.

Notation is collected in Table 2.

2 Related Work

Our problem sits between two literatures that have grown largely independently: flexible job shop scheduling with transportation, which decides what is processed where and in which order but abstracts the vehicles away, and conflict-free multi-robot routing, which resolves vehicle contention in space and time but receives its tasks from elsewhere. We review each, then the smaller body of work that combines them, and close by locating the gap this paper addresses.

2.1 Flexible job shop scheduling with transportation

The flexible job shop scheduling problem with processing machines and transportation vehicles (FJSP_PT) extends the FJSP by making material movement part of the schedule. Xin et al. [10] survey the field through 2023, cataloguing its assumptions, constraints, objectives and benchmarks, and classifying solution methods into exact, heuristic, meta-heuristic and swarm intelligence families. The dominant modelling convention is a constant travel-time matrix indexed by location pairs: a vehicle needs a fixed $t(a,b)$ to move a job from a to b, whatever else is happening in the shop. Vehicles are then a *capacity* resource, contending for jobs but never for space.

Work in this tradition has advanced rapidly on the combinatorial side. Meng et al. [6] give constraint programming models for FJSP with multiple AGVs that close 29 open benchmark instances and improve 27 best-known solutions, and pair them with a CP-assisted dual-population genetic algorithm. Qin et al. [7] combine a quality-diversity archive with Q-learning region selection and an AGV- and critical-path-guided local search. Fu et al. [2] integrate production and logistics decisions in a flexible assembly shop with an improved genetic algorithm. In all of them transport duration is read from a matrix, so a vehicle can never be delayed by another vehicle.

The survey is explicit that this is a limitation rather than a settled modelling choice, observing that "most scheduling problems currently have too many assumptions" and that these "can reduce problem complexity and even change the nature of the problem, e.g., when there are a sufficient number of transportation vehicles" [10]. It further notes that the arrival of jobs at machines is in practice affected by traffic congestion, and that the dependence implicit in the processing–transport two-phase flow must be handled before a schedule can be called feasible, let alone robust.

2.2 Conflict-free routing with exogenous tasks

The complementary literature treats contention as the object of study. In multi-agent path finding, and particularly its lifelong variant, agents must be routed without collision on a shared graph, and congestion is understood to degrade throughput well before capacity is reached. Zhang et al. [11] address this by optimizing a guidance graph whose edge weights steer agents away from bottlenecks and discourage head-on movement, improving the throughput of several lifelong solvers. In manufacturing, Kim and Kim [4] propose congestion functions, congestion penalties and congestion-aware dispatching rules for large AGV fleets, evaluated in a discrete event simulation.

Two features of this literature matter for us. First, the tasks are exogenous: their release times, origins and destinations are handed to the router, which is consequently never in a position to question the assignment that generated them. Second, where congestion is quantified, it is usually a proxy—local vehicle density in [4], edge weights learned offline in [11]—rather than the delay a specific reservation actually caused a specific task.

2.3 Integrating scheduling with conflict-free routing

A smaller body of work couples the two. On the exact side, van Os and Basten [9] formalize the conflict-free-transportation-constrained flexible job shop scheduling problem, which combines machine-eligible operations having location-dependent processing times with conflict-free vehicle routing, and solve it by logic-based Benders decomposition: a constraint programming master problem proposes schedules and an integer programming subproblem routes the vehicles, returning conflict-driven cuts. Most benchmark instances are closed to proven optimality, with makespan improvements of at least 10% over prior heuristics on about half of them. The approach is limited to the instance sizes exact decomposition can reach, and its authors point to using "information about infeasibility causes" more aggressively as the natural next step.

On the heuristic side the coupling is one-directional. Sun et al. [8] schedule with a hybrid genetic algorithm and then, in a second stage, resolve AGV conflicts with an AGV-encoded genetic algorithm and time-window Dijkstra; contention discovered in the second stage cannot revise the first. Li et al. [5] come closest to a loop, with a reinforcement learning scheduling layer, Gantt-derived task allocation, and a priority-based search path layer that feeds realized transport durations back into the timeline. Their own limitations section characterizes the result as operating "primarily as a two-stage sequential optimization strategy rather than a fully dynamic, real-time closed-loop system", relying on "a one-time feedback adjustment", and states that "the global optimality of the final solution is partially dependent on the quality of the initial schedule". The feedback also carries only time, so it can shift the schedule along the time axis but cannot revise an assignment; and their benchmarks are classical job shop instances with one eligible machine per operation, in which reassignment is not a decision at all.

2.4 What feedback carries, and how neighbourhoods are built

Critical-path reasoning is long established in job shop local search, and recent FJSP_PT work applies it with transport in view [7]. What changes when routing is conflict-free is the *content* of the diagnosis. A critical chain computed

Table 1. Positioning with respect to the closest lines of work. “Heterogeneity” means an operation has several eligible machines with machine-dependent processing times; “coupling” is how often, and in which direction, routing outcomes reach the scheduling decision.

Line of work	Ref.	Heterogeneity	Vehicle contention	Coupling	Feedback content
FJSP_PT metaheuristics	[2, 6, 7]	yes	constant matrix	—	—
Lifelong MAPF	[11]	—	conflict-free	tasks exogenous	edge weights
Congestion-aware dispatch	[4]	no	density proxy	one-way	congestion penalty
Two-stage FJSP + routing	[8]	yes	conflict-free	one-way	none
RL scheduling + PBS	[5]	no	conflict-free	one-time refinement	arrival times
Exact LBBD	[9]	yes	conflict-free	master–subproblem	conflict-driven cuts
This paper		yes	conflict-free	every candidate	contention certificates

against a travel-time matrix can attribute a late start to a slow predecessor or a busy machine, but it cannot attribute it to a corridor, because in that model corridors do not exist as contended resources. Once occupancy is reserved over time windows, a late start can be traced to a specific segment and a specific interval, and to the task that reserved it first—which is the information an operator needs in order to know whether to move an operation elsewhere or merely to move it earlier. This is the heuristic counterpart of the conflict-driven cut in [9]: structured evidence about why the lower level could not do better, returned to the level that can act on it. The survey identifies precisely this as an open direction, noting that most algorithms are “further adaptation and improvement of FJSP-solving algorithms, lacking in-depth analysis of problem characteristics and operator design based on problem features” [10].

2.5 Experimental practice

Xin et al. [10] also fault the field’s instances: they “differ primarily in scale, lacking specialized design for different values of key problem parameters”, and the survey singles out the ratio of mean transport time to mean processing time, $\bar{T}_t/\bar{T}_p$, as a quantity highly likely to affect how well an algorithm performs, calling for enriched test sets. To this we would add a second parameter that only becomes visible once routing is explicit: how much of an instance’s congestion is assignment-relevant. Congestion at a load/unload exit is crossed by every job under every assignment and therefore rewards no decision, while congestion on corridors serving a subset of machines can be routed around. Two instances with identical $\bar{T}_t/\bar{T}_p$ can differ completely in how much leverage a scheduling mechanism has, which we found sufficient to reverse the apparent value of a mechanism.

2.6 Positioning

Table 1 summarizes the comparison. The problem class we work in is not new; it is the one formalized in [9], and we adopt it. What is open is the heuristic regime, where instances outgrow exact decomposition and where the coupling between scheduling and routing is still either absent, one-way, or applied once to a single incumbent. Our contribution is the content of the interlayer signal and the use made of it: typed contention certificates, produced for every candidate the search evaluates, that name the operations to reassign or to stagger.

3 Problem Formulation

3.1 Problem statement

An instance consists of n jobs, N_M robot arms and N_A automated guided vehicles operating on a shop floor represented as a graph. Job j has a fixed chain of n_j operations $O_{j1} \to \cdots \to O_{jn_j}$; operation O_{ji} may be processed on any arm in its eligible set $\Omega_{ji} \subseteq M$, taking t^P_{jim} time units on arm m, a quantity that depends on the arm chosen. Arms occupy

Table 2. Notation.

Symbol	Meaning
$J = \{1, \ldots, n\}$, O_{ji}	jobs; i-th operation of job j, $i \leq n_j$
$M = \{1, \ldots, N_M\}$, $\Omega_{ji} \subseteq M$	robot arms; eligible arms for O_{ji}
t^P_{jim}	processing time of O_{ji} on arm $m \in \Omega_{ji}$
$K = \{1, \ldots, N_A\}$	identical unit-load vehicles
$G = (V, E)$, τ_e	shop graph; traversal time of corridor $e \in E$
v_0, $\mathrm{loc}(m)$	load/unload station; node of arm m
$t^*(a, b)$	shortest-path time from a to b in G, ignoring contention
$\delta_{\mathrm{ret}} \in \{0, 1\}$	whether finished jobs return to v_0 (baseline 1)
S_{ji}, C_{ji}, A_{ji}	start, completion, and material arrival time of O_{ji}
$C_{\max}$	makespan

fixed nodes of the graph and jobs are moved between them by the vehicle fleet. Solving an instance means fixing four groups of decisions simultaneously:

(i) assign each operation O_{ji} to an arm $m \in \Omega_{ji}$;
(ii) sequence the operations assigned to each arm;
(iii) assign the resulting transport tasks to vehicles and sequence each vehicle's tasks;
(iv) route every vehicle movement over the graph, conflict-free in time and space, with waiting permitted.

The objective is to minimize the makespan $C_{\max}$. Table 2 collects the notation.

3.2 Assumptions

A1. Static instance, complete information. All n jobs are available at v_0 at $t = 0$; there are no release or due dates. Processing data, the layout G and vehicle states are known when planning.

A2. Jobs. Each job has a fixed precedence chain. Processing and transport are non-preemptive, jobs are unsplittable, and there is no exogenous precedence between different jobs.

A3. Arms: partially flexible and time-heterogeneous. Each O_{ji} may run on any $m \in \Omega_{ji}$, with $|\Omega_{ji}| \geq 2$ for most operations and total flexibility $\Omega_{ji} = M$ as a special case. Processing times are machine-dependent in the unrelated sense: t^P_{jim} and $t^P_{jim'}$ need bear no relation for $m \neq m'$. An arm processes one operation at a time, does not fail within the horizon, and sequence-dependent setups are absorbed into t^P_{jim}.

A4. Buffers and handling. Input and output buffers at each arm, and storage at v_0, have ample capacity, so no blocking occurs. Loading and unloading take negligible time and do not seize the arm.

A5. Vehicles. The N_A vehicles are identical, unit-load, and move at constant speed whether laden or empty; acceleration and battery are not modelled. A transport task is a single move—from v_0 to the first arm, between consecutive arms when they differ, or from the last arm back to v_0 when $\delta_{\mathrm{ret}} = 1$—and any vehicle may execute any task, so consecutive moves of one job may be relayed across vehicles. After unloading, a vehicle is not forced back to v_0; it repositions according to its own task chain, and its departure may be deferred by the router.

A6. Network and conflict-freeness. G is strongly connected with known constant traversal times τ_e. A corridor is an *exclusive* resource: both directions of e share a single reservation, so at most one vehicle occupies e at any instant, which rules out head-on conflicts and same-direction overtaking alike. Occupancy is a half-open interval

$[t,\ t+\tau_e)$, so a second vehicle may enter exactly at $t+\tau_e$. Nodes carry parking berths of ample capacity that are separate from the traversal resource; waiting therefore happens at berths and is always permitted.

A7. **Deadlock freedom by construction.** Routes are planned by time-window reservation on the corridor resource, and a new route may only occupy free windows. Together with A4 and A6 this makes every assignment, sequence and task chain decodable into a conflict-free, deadlock-free schedule of finite makespan, so no repair of infeasibility is ever required.

A8. **Out of scope.** Machine and vehicle failures, finite buffers and blocking, battery charging, and dynamic job arrivals are left to future work.

Assumptions A1, A2, A4 and A5 are standard in the FJSP_PT literature [10]. The choices that shape this paper are A3 (machine-dependent times on partially flexible arms), A6 (contention as an explicit exclusive-resource constraint rather than a constant travel matrix) and the resulting fact that transport durations are outputs of the routing decision.

3.3 Decisions and derived transport tasks

Let $x_{jim} \in \{0,1\}$ indicate that O_{ji} is assigned to arm m, and write $\mathrm{m}(j,i)$ for the arm so chosen and $d_{ji} = \mathrm{loc}(\mathrm{m}(j,i))$ for its node. For $\delta_{\mathrm{ret}} = 1$ we append to each job a *pseudo-operation* O_{j,n_j+1} with zero processing time located at v_0, which represents the return of the finished job; this keeps the makespan definition uniform.

Unlike vehicle routing problems, the transport tasks are not part of the input. They are induced by the assignment:

$$\mathcal{T}(x) = \Big\{(j,i)\ :\ d_{j,i-1} \neq d_{ji}\Big\}, \qquad d_{j0} \equiv v_0, \tag{1}$$

where task (j,i) moves job j from $d_{j,i-1}$ to d_{ji} and becomes ready at $r_{ji} = C_{j,i-1}$ (with $C_{j0} = 0$). Consecutive operations placed on the same arm generate no task at all. Hence both the number of transport tasks and their endpoints are consequences of decision group (i): an assignment that keeps a job on one arm moves it twice, while one that changes arm at every step moves it $n_j + 1$ times. Task assignment to vehicles is expressed by $y_{tk} \in \{0,1\}$, $\sum_{k\in K} y_{tk} = 1$ for every $t \in \mathcal{T}(x)$, and the tasks allotted to a vehicle are totally ordered.

3.4 Timing and feasibility

Write $p_{ji} = \sum_{m\in\Omega_{ji}} x_{jim} t^{P}_{jim}$ for the realized processing time. Assignment, precedence and arm capacity give

$$\sum_{m\in\Omega_{ji}} x_{jim} = 1, \qquad \forall j,i, \tag{2}$$

$$C_{ji} = S_{ji} + p_{ji}, \qquad \forall j,i, \tag{3}$$

$$S_{ji}\ \geq\ A_{ji}, \qquad \forall j,i, \tag{4}$$

$$S_{j'i'}\ \geq\ C_{ji}\ \ \vee\ \ S_{ji}\ \geq\ C_{j'i'} \qquad \forall (j,i) \neq (j',i') \text{ on the same arm}, \tag{5}$$

where A_{ji} is the time the job is physically present at d_{ji}. If $(j,i) \notin \mathcal{T}(x)$ the job is already there and $A_{ji} = C_{j,i-1}$. Otherwise A_{ji} is produced by the vehicle that carries task (j,i): with a^{E}_{ji} the arrival of that vehicle at the pickup node after its empty leg,

$$\ell_{ji} = \max\big(a^{\mathrm{E}}_{ji},\ r_{ji}\big), \qquad A_{ji} = \text{arrival of the laden leg departing at } \ell_{ji}, \tag{6}$$

so a vehicle may wait for the job or the job for the vehicle. Each leg, empty or laden, is a walk $(e_1, \ldots, e_q)$ in G with corridor entry times $\theta_1, \ldots, \theta_q$ satisfying

$$\theta_1 \;\geq\; \sigma, \qquad \theta_{l+1} \;\geq\; \theta_l + \tau_{e_l}, \quad l = 1, \ldots, q-1, \tag{7}$$

with σ the time the leg becomes available to start; the inequalities are slack exactly when the vehicle waits at a berth, and the leg ends at $\theta_q + \tau_{e_q}$. Finally, contention:

$$\big[\theta,\ \theta + \tau_e\big) \;\cap\; \big[\theta',\ \theta' + \tau_e\big) \;=\; \emptyset \tag{8}$$

for every two distinct occupancies of the same corridor e, irrespective of direction or of which vehicles are involved. The objective is

$$\min\ C_{\max}, \qquad C_{\max} = \max_{j \in J}\ C_{j,\, n_j + \delta_{\text{ret}}}. \tag{9}$$

Two consequences are worth recording, because the algorithm of Section 4 is built on them. First, in any semi-active schedule, (4) and (5) tighten to

$$S_{ji} = \max\big(A_{ji},\ F_{\mathrm{m}(j,i)}\big), \tag{10}$$

where F_m is the completion time of the operation preceding O_{ji} on arm m. The start time is therefore governed by exactly one of two causes, and which of the two attains the maximum is well defined; this is what makes the later attribution of a delay to a cause meaningful rather than a heuristic guess. Second, (8) is the only constraint in the model that couples distinct jobs through space rather than through a shared machine.

3.5 Relation to FJSP_PT

Deleting (8) makes every leg independent, so each one may follow a shortest path and (7) holds with equality throughout. Transport durations then collapse to the constant matrix $t^*(a, b)$ and the model becomes the standard flexible job shop scheduling problem with transportation. The entire difference between the two problems is thus one family of constraints—and with it the fact that the duration of a move is no longer a parameter but a variable determined jointly by the routes of all vehicles. The same relaxation supplies the natural optimistic estimate t^* used later both as a lower bound and, in the open-loop baselines, as the dispatcher's estimate of arrival.

3.6 Complexity

PROPOSITION 3.1. *The problem defined by (2)–(9) is strongly NP-hard, and remains so when $\Omega_{ji} = M$ for all operations.*

PROOF. Restrict to instances in which $\tau_e = 0$ for all $e \in E$, each arm is joined to v_0 by its own dedicated corridor, and $N_A = n$. Every leg then takes zero time and occupies the half-open interval $[t, t) = \emptyset$, so (8) is vacuous and no vehicle can delay another; moreover each job owns a vehicle, so (6) never binds and $A_{ji} = C_{j,i-1}$. What remains is (2)–(5) with $A_{ji} = C_{j,i-1}$, which is exactly the flexible job shop scheduling problem with unrelated machines. Two of its special cases are already strongly NP-hard: taking $|\Omega_{ji}| = 1$ throughout leaves the classical job shop problem, strongly NP-hard for three machines [3]; taking $\Omega_{ji} = M$ and $n_j = 1$ leaves the minimization of makespan on unrelated parallel machines, which contains scheduling on identical parallel machines and is strongly NP-hard for a variable number of machines. Each restriction is polynomial, so the general problem is strongly NP-hard, and the second case shows that total flexibility does not help. ◻

The reduction deliberately switches contention off, which shows that hardness is inherited from the scheduling side alone; the routing side adds to it rather than replacing it, since the routing subproblem obtained by freezing decisions (i)–(iii) is itself a conflict-free multi-agent routing problem in which release times and endpoints are chained across tasks. Exact methods are available for small instances—van Os and Basten [9] close most instances of the same problem class by logic-based Benders decomposition—and we use them to frame, not to compete with, the heuristic regime this paper targets.

3.7 Characterizing an instance

Because the effect of the mechanisms we study depends on instance structure rather than instance size, every instance is recorded with the following descriptors. The first four are conventional; the survey of Xin et al. [10] singles out the first as likely to govern how well an algorithm performs.

- $\bar{T}_t/\bar{T}_p$: mean shortest-path travel time between arm nodes divided by mean processing time, the relative weight of moving versus processing.
- Heterogeneity H: the mean, over operations, of the coefficient of variation of $\{t^P_{jim} : m \in \Omega_{ji}\}$. $H = 0$ makes eligible arms interchangeable in duration and switches off the reassignment trade-off.
- Flexibility $F = \mathrm{mean}_{ji} |\Omega_{ji}|/N_M$: the room a reassignment has to manoeuvre.
- N_A/N_M: whether the bottleneck is likely to sit on the transport or the processing side.

The remaining descriptors are specific to the present setting, and exist because congestion as such is not the right quantity: what matters is how much of it a scheduling decision can avoid. Let $\mathrm{cut}(v_0)$ be a minimum-cardinality set of corridors whose removal disconnects v_0 from every arm.

- Funnel share $\varphi \in [0, 1]$: the fraction of the time of a typical $v_0 \longrightarrow$ arm trip spent on corridors common to *all* shortest $v_0 \longrightarrow$ arm paths. Such corridors are crossed by every job under every assignment, so the delay they cause is invariant to decisions (i)–(iii); a large φ dilutes the signal any feedback mechanism could exploit.
- $|\mathrm{cut}(v_0)|$, the funnel width, which upper-bounds how many vehicles can leave the station concurrently; a value of 1 means a single-point funnel.
- The far-group cut: the minimum number of corridors isolating the arms whose distance to v_0 exceeds the median. Cutting a single arm is uninformative, since its own spur always yields 1; the group cut measures the capacity of the deep bottleneck that assignment decisions actually contend for.

Finally, each instance carries a zero-cost composite lower bound $\mathrm{LB} = \max(\mathrm{LB}_{\mathrm{chain}}, \mathrm{LB}_{\mathrm{load}}, \mathrm{LB}_{\mathrm{cut}})$, whose three components relax different parts of the model and do not dominate one another. $\mathrm{LB}_{\mathrm{chain}}$ takes, per job, the shortest inbound trip plus $\sum_i \min_{m\in\Omega_{ji}} t^P_{jim}$ plus the shortest return, relaxing away inter-arm moves and all queueing. $\mathrm{LB}_{\mathrm{load}}$ is $\sum_{j,i} \min_m t^P_{jim}/N_M$, relaxing precedence and transport. $\mathrm{LB}_{\mathrm{cut}}$ exploits the funnel: with $\delta_{\mathrm{ret}} = 1$ each job crosses $\mathrm{cut}(v_0)$ twice, and distributing the $X = 2n$ crossings over the cut so as to minimize the busiest corridor gives

$$C_{\max} \;\geq\; X \Big/ \sum_{e\in\mathrm{cut}(v_0)} \tau_e^{-1}. \tag{11}$$

The bound is loose—observed gaps to the best known solutions run from roughly 35% to 61%—so it serves for order-of-magnitude checks and as a regression guard rather than as a quality certificate.

4 Closed-Loop Bilevel Framework

4.1 Overview: two feedback paths

The framework splits the four decision groups of Section 3 across two layers. The upper layer searches over assignment and sequencing with a memetic algorithm; the lower layer turns a given set of transport tasks into conflict-free routes by time-window reservation. What distinguishes it from a two-stage or one-shot-refinement scheme is not the split but what crosses it, in both directions and how often.

Evaluation path. The fitness of *every* individual in *every* generation is the makespan obtained after the lower layer has routed all of its transport tasks conflict-free, yielding delays included. The upper layer never sees an optimistic estimate, so it never optimizes a proxy.

Decision path. Having resolved contention, the lower layer knows precisely who blocked whom, on which corridor, and over which interval. It returns that structure—a *contention certificate*—rather than a scalar. Certificates name the operations that two families of operators then act upon: reassignment, which changes *where* an operation is processed, and staggering, which changes *when* a movement is initiated.

The evaluation path makes the objective honest; the decision path makes the neighbourhood informed. Both are required, and Section 5 separates their contributions by closing one link at a time. Section 4.4 argues why the decision path carries structure rather than prices, which is the design choice the paper's title refers to.

4.2 Lower layer: conflict-free routing

4.2.1 Reservation table. The only contended resource is the corridor (assumption A6), so the table maps each corridor to a time-ordered list of half-open occupancies $[t, t + \tau_e)$ tagged with the occupying vehicle and task. It supports querying the earliest feasible entry at or after a given time, committing an occupancy, releasing all occupancies of a task, and checkpoint/rollback of the whole table. Nodes are not reserved: crossing a node takes zero time, and waiting always happens at berths, which assumption A6 separates from the traversal resource. A single resource type is therefore enough, which keeps both the router and the independent feasibility checker small.

4.2.2 Time-window Dijkstra. A leg is routed by label-setting search in which a label (u, t) means "standing at a berth of node u at time t" and costs t (Algorithm 1). Relaxing an outgoing corridor asks the table for the earliest entry $t' \geq t$, which is exactly where waiting enters: the gap $t' - t$ is time spent at the berth of u, and the slack in (7) is generated by the search rather than modelled separately. Committed routes are never revoked within one decoding, so planning follows the order in which the decoder generates tasks—prioritized planning with a deterministic priority.

LEMMA 4.1 (EARLIEST ARRIVAL SUFFICES). *Keeping only the earliest arrival time at each node yields a minimum-arrival-time leg with respect to the current reservation table.*

PROOF. Berths have ample capacity and do not occupy the traversal resource, so a vehicle standing at u from time t can reproduce any behaviour available to a vehicle that reaches u at $t' > t$, by simply waiting until t'. Arrival time is thus a dominance criterion: an earlier label is never worse. Every corridor relaxation is non-decreasing in t, so the search is label-setting and the first extraction of the goal is optimal. □

The lemma is what allows a plain Dijkstra label rather than the interval labels of safe-interval planning, and it is fragile in an instructive way: introduce a second criterion, such as a price paid for occupancy, and dominance is no longer a total order, forcing a multi-label Pareto search. We return to this in Section 4.4.

Algorithm 1 Route: conflict-free leg planning

Require: origin s, destination g, earliest departure t_0, reservation table R, flag *commit*
Ensure: arrival time and the sequence of corridor occupancies

push label (s, t_0) into a priority queue keyed by time
while queue not empty **do**
    pop the label (u, t) of least time
    if $u = g$ **then**
        backtrack to obtain the occupancies; if *commit*, write each of them into R; **return** arrival t
    end if
    for all corridors $e = (u, v)$ leaving u **do**
        $t_{\text{in}} \leftarrow R.\textsc{EarliestEntry}(e, t, \tau_e)$ ▷ waits at the berth of u until t_{in}
        relax label $(v,\ t_{\text{in}} + \tau_e)$
    end for
end while

Because reservations are finite in number and bounded in time, and G is strongly connected, a route always exists: in the worst case the vehicle waits until every existing occupancy has expired and then travels unobstructed. This is the observation that makes Proposition 4.2 below true, and it is why no deadlock repair is needed. One call costs $O(|E|\, W \log |V|)$ with W the mean number of reserved intervals per corridor.

Running the same search with an empty table over all pairs of pickup and delivery nodes yields the contention-free matrix t^* of Section 3. It is used in three places: as the dispatcher's estimate in the open-loop configurations, as the proximity term of the reassignment operator, and as the transport time of the two-stage baseline.

4.3 Upper layer: encoding and decoding

4.3.1 Chromosome. With N operations in total, an individual is a pair of segments: a machine assignment vector of length N, whose k-th entry holds an arm drawn from Ω_{ji} for the k-th operation in a fixed enumeration, and an operation sequence, a permutation with repetition in which job j appears n_j times and its x-th occurrence denotes O_{jx}. The sequence is a global priority order, and the relative order of operations sharing an arm is read off from it.

The return trip is folded into the same representation. For $\delta_{\text{ret}} = 1$ each job contributes one extra occurrence corresponding to the pseudo-operation of Section 3, whose arm is fixed to v_0 and whose processing time is zero. Return moves therefore compete for vehicles in the same stream as ordinary moves and are ordered by the same segment, with no special case in the decoder.

Task-to-vehicle assignment is deliberately *not* encoded; it is decided during decoding (Section 4.3.3). A third segment for it exists in the literature, but it enlarges the search space and forfeits the guarantee that every chromosome decodes, which would require repair operators.

4.3.2 Decoder. The decoder (Algorithm 2) is the evaluation path. It sweeps the operation sequence left to right, maintaining for each arm its free time, for each job its current node and ready time, and for each vehicle its current node and available time. When an operation's arm differs from the job's current location, the transport task of (1) is created, a vehicle is chosen, and two legs are routed against the live reservation table: an empty leg to the pickup and a laden leg to the destination, joined by (6). The operation then starts according to (10), and the decoder records which of the two terms attained the maximum—the `bind` field, on which all later attribution rests.

Two structural properties follow. Reservations are made in the order the sweep generates tasks, so one chromosome always yields one timetable: evaluation is deterministic, which the equal-budget protocol of Section 5 relies on. And

Algorithm 2 DECODE: fitness by complete conflict-free simulation

Require: instance, network, assignment x, operation sequence
Ensure: makespan, timetable, per-corridor yielding statistics

$F_m \leftarrow 0$ for all arms; $\text{pos}_j \leftarrow v_0$, $\text{ready}_j \leftarrow 0$ for all jobs
$\text{loc}_k \leftarrow v_0$, $\text{avail}_k \leftarrow 0$ for all vehicles; clear the reservation table
for all operations O_{ji} in sequence order (pseudo-operations included) **do**
  $m \leftarrow$ arm of O_{ji} under x; $d \leftarrow \text{loc}(m)$
  if $\text{pos}_j = d$ **then**
    $A_{ji} \leftarrow \text{ready}_j$ ▷ no move is generated
  else
    choose vehicle k (Section 4.3.3)
    $a^{\text{E}} \leftarrow \textsc{Route}(\text{loc}_k, \text{pos}_j, \text{avail}_k)$; $\ell \leftarrow \max(a^{\text{E}}, \text{ready}_j)$
    $A_{ji} \leftarrow \textsc{Route}(\text{pos}_j, d, \ell)$; $\text{loc}_k \leftarrow d$; $\text{avail}_k \leftarrow A_{ji}$
    accumulate yielding time per corridor
  end if
  `bind` $\leftarrow$ `machine` if $F_m > A_{ji}$ else `arrive`
  $S_{ji} \leftarrow \max(A_{ji}, F_m)$; $C_{ji} \leftarrow S_{ji} + p_{ji}$; $F_m \leftarrow C_{ji}$; $\text{pos}_j \leftarrow d$; $\text{ready}_j \leftarrow C_{ji}$
end for
return $C_{\max} = \max_{j,i} C_{ji}$ and the timetable

state flows strictly forward, so a neighbour that perturbs the sequence only after position x leaves every event and every reservation before x unchanged—the basis for incremental evaluation, which we note as an available speed-up that the present implementation does not exploit.

PROPOSITION 4.2 (EVERY CHROMOSOME DECODES). *For any assignment and any operation sequence, Algorithm 2 terminates and returns a conflict-free, deadlock-free schedule of finite makespan. No repair operator is ever invoked.*

PROOF. The sequence is by construction a topological order of the precedence chains, since the x-th occurrence of a job denotes its x-th operation; hence ready_j is known whenever it is read. Ample buffers (A4) mean an operation is never blocked for want of storage, so (10) always yields a finite start once the material has arrived. Each ROUTE call returns in finite time by the argument following Algorithm 1, and it only occupies windows the table reports free, so (8) holds by construction. The sweep performs $N + n\delta_{\text{ret}}$ iterations, each scheduling finitely many events at finite times. □

4.3.3 Dispatching. Choosing the vehicle is the one place where an estimate could replace a fact, and we treat it as a closable link rather than a fixed design. The open-loop rule scores each vehicle by $\max(\text{avail}_k + t^*(\text{loc}_k, \text{pickup}),\ \text{ready}_j) + t^*(\text{pickup}, \text{destination})$ and never consults the reservation table, so it may pick a vehicle that is close on the map but walled in by traffic. The closed variant, which is the default, probes instead: for each candidate it routes the empty leg with commitment—necessary, or the two legs could be planned into the same corridor window and yield an unrealizable optimism—routes the laden leg without commitment, records the true delivery time, and rolls the table back. It costs $2N_A$ route calls per task against none, roughly a fivefold slowdown per evaluation, which is precisely why Section 5 compares the two only under an equal wall-clock budget.

4.4 What should cross the interface?

Both layers being fixed, the design question is what the lower layer reports. Three answers are available, and the choice among them is the substance of the paper.

Time alone. The lower layer returns realized durations, which is what the evaluation path already does. This makes the objective correct but leaves the search blind: the upper layer learns that a solution was slow, not which decision made it slow, so its neighbourhoods remain undirected. Methods that route once after scheduling stop here.

Prices. The natural next step, by analogy with Lagrangian and column-generation coordination, is to price the contended resource: attach a shadow price $\pi(e, b)$ to each corridor–time slot, let the router minimize arrival $+\ \theta \cdot$ price over a Pareto frontier, and let reassignment pay for the access it needs. We implemented this fully—a definitional estimator that perturbs one slot's capacity and re-solves, and a cheap surrogate refreshed each generation—and it is *systematically harmful*, monotonically so in θ (Section 5).

The mechanism is worth stating because it generalizes. A delay caused by contention is *already* internalized: the yielding vehicle waits, the wait enters its own arrival time, propagates through (10) into the makespan, and is counted by the fitness. Charging the occupant a price for the same occupancy counts it twice. The vehicle then detours or defers to avoid a cost that has already been paid, accepting a delay that is first-order and certain in order to avert an externality that is second-order and uncertain. What genuine externality remains falls only on tasks not yet routed—the sweep is ordered, so an early reservation can only obstruct a later one—and that, too, is already visible in the makespan of the fully decoded solution. Pricing therefore adds distortion without adding information. The inference is architectural rather than parametric: *in any framework whose fitness is a complete conflict-free decoding, adding a congestion penalty to the lower layer's objective is likely to hurt.*

Contention certificates. The remaining option is to pass structure. When the router yields, it knows the corridor, the interval, and the identity of the occupying task; the decoder retains yielding *events* rather than only their total, so this information survives to the end of the evaluation. A certificate is the triple (corridor, interval, opposing task), and it is used to decide which operations an operator should touch. Crucially it does not enter any objective: the lower layer still minimizes arrival time, the upper layer still minimizes makespan, and only the *construction* of the neighbourhood changes. No quantity is counted twice, because no quantity is added. This is the interface we adopt, and the next subsection turns it into operators.

4.5 Contention-guided local search

Each generation, the best 10% of the population undergo local search.

4.5.1 Attributing the critical chain. Tracing back from the event that attains the makespan produces a chain, but a chain of operations alone cannot say why a start was late. When `bind = arrive`, the operation waited for material, and the cause may be an upstream operation, an unavailable vehicle, or traffic—three situations calling for three different remedies. We therefore trace through the transport legs as well and label every link (Table 3). Laden-leg yielding is always recorded as a corridor link, since it lengthened the arrival directly. On an empty leg the split between vehicle and corridor is decided by the share of yielding in the leg's total elapsed time, against a threshold of 0.5.

Three queries are derived from the labelled chain: the real operations on it, which the reassignment operator draws candidates from; the corridor–time slots on it; and, for a given corridor and interval, the tasks that occupied it and thereby caused the yielding—the certificate proper.

4.5.2 Two operator families. Congestion admits two remedies, and the framework provides one operator for each.

Table 3. Labels attached to the links of the critical chain, and the operator each one calls for. Only the last carries a corridor and an interval, and only it can name an opponent.

Label	Condition	Remedy
`operation`	a real operation occupies its arm	–
`machine`	`bind` = `machine`, blocked by the previous operation on that arm	resequencing (generic mutation)
`upstream`	vehicle arrived first, material was not ready	recurse to $O_{j,i-1}$
`vehicle`	material ready, vehicle late, little of the delay spent yielding	change vehicle
`corridor`	yielding on a named corridor over a named interval	reassign or stagger

Reassignment changes where. For at most $L = 5$ real operations on the critical chain, each alternative arm $m' \in \Omega_{ji} \setminus \{m\}$ is scored by

$$\text{score}(m') = t^*\big(\text{pos}_{j,i-1}, \text{loc}(m')\big) + t^P_{jim'}, \tag{12}$$

the sum of a proximity term and a duration term, both in time units, and the best-scoring arm produces one neighbour differing in a single gene. The two terms are exactly the trade-off the paper is about: a nearer arm may be slower, and (12) is what lets the search buy proximity with processing time when the route to the fast arm is contended.

An earlier version of this score added λ times the accumulated yielding time at the candidate arm. That term is a horizon-wide, fleet-wide total which grows with instance size while the other two terms remain at the scale of a single operation; on a 240-operation instance it dominates the score, and the operator degenerates into "always choose the least congested arm", destroying the very trade-off it was meant to express. We report this because λ is not comparable across instances and any sensitivity analysis over it would be meaningless.

Staggering changes when. Take the longest `corridor` link on the chain, and query the certificate for the tasks that held that corridor over that interval. Two directed neighbours follow: move the blocked operation earlier in the sequence, so its movement is initiated into a freer window, or move a single opposing operation later, so the two no longer coincide. Only one opponent is displaced per round, keeping the neighbourhood small and pointed. Shifting an occurrence in the sequence exchanges it with a gene of a *different* job, so intra-job order is preserved automatically and the result is still a valid permutation with repetition; by Proposition 4.2 it therefore remains decodable.

The neighbourhood thus contains at most $L + 2 = 7$ candidates, every one of them named by the lower layer. This is the concrete difference from a generic mutation, which would sample the same space uniformly at random.

4.5.3 Acceptance. The first neighbour that improves the makespan replaces the incumbent immediately; the chain is then re-extracted from the new incumbent and the round repeats, up to three rounds. A round that produces no improvement ends the search (Algorithm 3). Re-extraction matters: after a successful move the binding cause of the makespan usually shifts, so a chain computed once at the outset would send later rounds after a bottleneck that no longer exists.

4.6 The surrounding genetic algorithm

The population-based layer is deliberately conventional, so that any measured gain is attributable to the mechanisms above rather than to operator tuning. The initial population is random—arms drawn uniformly from Ω_{ji}, sequences shuffled—save for two seeded individuals, one assigning every operation its fastest arm and one balancing load greedily. Selection is binary tournament with five elites retained. Crossover applies precedence-preserving order crossover to the sequence segment and uniform crossover to the assignment segment; mutation swaps two positions of the sequence or

Algorithm 3 ContentionGuidedSearch

Require: chromosome c with its decoded result r

for *round* = 1 **to** 3 **do**
  $\mathcal{N} \leftarrow \textsc{Reassign}(c, r) \ \cup\ \textsc{Stagger}(c, r)$ ▷ $\leq L + 2$ neighbours, all named by the certificate
  improved ← **false**
  for all $c' \in \mathcal{N}$ **do**
    $r' \leftarrow \textsc{Decode}(c')$
    if $r'.C_{\max} < r.C_{\max}$ **then**
      $c \leftarrow c'$; $r \leftarrow r'$; *improved* ← **true**; **break**
    end if
  end for
  if not *improved* **then**
    break
  end if
end for
return c, r

moves one operation to another eligible arm. Fitness is the decoded makespan. These choices are standard and are not claimed as contributions.

4.7 Cost, and what is not claimed

One fitness evaluation sweeps $N + n\delta_{\text{ret}}$ events and, under closed-loop dispatching, spends $2N_A$ route calls per move, giving $O\big((N + n)\, N_A\, |E|\, W \log |V|\big)$ per individual; the open-loop rule removes the factor N_A at the cost of an estimate. Local search adds up to $3(L + 2)$ further decodings for each of the individuals it touches. The consequence is that mechanisms in this framework differ in cost by more than an order of magnitude—measured at 0.42 ms per evaluation for the two-stage configuration against 15.7 ms for the closed loop—so comparisons at equal generation counts are not meaningful, and Section 5 equalizes wall-clock time instead.

Finally, a disclaimer that the structure of the framework invites. Bilevel schemes that exchange multipliers or prices can sometimes be shown to converge to a fixed point; ours cannot, and we do not claim it. The lower layer is prioritized rather than jointly optimal, the upper layer is a stochastic search over a non-convex combinatorial space, and the interface transmits evidence rather than a quantity that could serve as a Lyapunov function. What Proposition 4.2 and Lemma 4.1 guarantee is narrower and, for a heuristic, more useful: every candidate the search examines is feasible, its evaluation is exact with respect to the conflict model of Section 3, and it is reproducible. Solution quality is an empirical question, which the next section addresses under a protocol designed to keep it honest.

5 Experimental Evaluation

The experiments answer four questions. Does closing the loop help at all, once the comparison is made fair? Which link in the chain contributes what? Does the benefit depend on the *structure* of the congestion, as Section 3.7 predicts it must? And does pricing the interface help, as the coordination literature would suggest?

5.1 Instances

Public FJSP_PT benchmarks specify travel as a matrix of location-pair durations, which is precisely the assumption this paper removes; they carry no aisle network, so vehicle contention cannot be defined on them. We therefore generate a

Table 4. The instance cells used in Sections 5.4–5.6. All share 8 jobs, 4 arms, 4 vehicles, $F = 0.6$ and $\bar{T}_t/\bar{T}_p \approx 1.0$; `high` and `funnel` differ only in the width of the station cut.

Structure	H	φ	cut	far cut	LB	binding term
[TBD: fill from clbs/experiments/instances.csv**]**						

controlled family in which the network is explicit, and in which the quantities that Section 3.7 identifies as decisive can be varied one at a time.

Every instance has 8 jobs, 24 real operations, 4 arms and 4 vehicles on a 10-node, 10-corridor layout, with $\bar{T}_t/\bar{T}_p$ calibrated to 1.0 so that transport and processing carry comparable weight, and flexibility fixed at $F = 0.6$. Two factors are then crossed.

Heterogeneity $H \in \{0, 0.15, 0.3, 0.5\}$ is the mean within-row coefficient of variation of the processing times. It is applied as a multiplicative perturbation around a per-operation base time, so changing H alters how much arms differ from one another without altering the average duration of the work.

Congestion structure takes four settings that change network capacity while leaving distances, processing times and arm placements untouched. The two we rely on most, `high` and `funnel`, differ in exactly one respect: the minimum cut separating the load/unload station from the shop is two corridors in the first case and one in the second. At equal H and equal seed the two instances are otherwise identical field by field, which the test suite asserts. This is what makes the comparison in Section 5.5 a controlled one rather than a correlation: `funnel` adds congestion that no assignment and no sequence can route around, and nothing else.

Table 4 lists the structural indicators and the composite lower bound of Section 3.7 for the cells used below. The funnel share φ confirms the intent of the design: on `high` only a small part of a typical inbound trip lies on corridors every job must cross, whereas on `funnel` the single-corridor cut forces every job through the same resource twice.

5.2 Comparison protocol

Two pitfalls make a naive comparison in this setting almost meaningless, and we report the protocol in detail because we walked into both of them before adopting it.

Equal computation, not equal generations. The arms of the ablation chain do not evaluate the same object. The two-stage baseline searches against the constant matrix t^*, so one of its evaluations is a decoding with table lookups; the closed loop routes every candidate conflict-free; the priced variant additionally runs a multi-label search. Measured cost per evaluation spans two orders of magnitude (Fig. 2). Comparing at equal generation counts therefore hands the expensive arm tens of times more computation, and an apparent mechanism gain may be nothing but a larger budget. We equalize wall-clock time instead: for each instance, the full method is run once under its natural stopping rule, and its running time becomes the budget shared by every arm on that instance. Early-stopping thresholds are relaxed so that the budget is actually consumed, and each run reports whether it stopped on the budget or converged first.

Equalizing time is not neutral either—it hands the cheap surrogate model tens of times more search—so neither budget can be presented alone. We report the cost per evaluation and the number of evaluations achieved for every arm, so that any difference in search effort is visible rather than hidden.

Enough seeds, and paired tests. Single-seed readings in this problem are worthless: on one instance we observed a range of 11 makespan units across three seeds of the *same* configuration, which exceeds every mechanism effect

Table 5. The seven configurations. Each row of the chain adds exactly one closed link to the row above it.

#	Configuration	What it does
1	dispatch rule	shortest processing time, one conflict-free decoding, no search
2	two-stage	the same search against t^*, then routes repaired once afterwards
3	evaluation loop	fitness is the true routed makespan; no feedback into the neighbourhood
4	+ certificates	contention-guided reassignment and staggering; dispatch still open-loop
5	+ closed dispatch	dispatching probes the reservation table
6	**full**	rows 3–5 together: the method of Section 4
7	+ pricing	the full method with corridor prices, $\theta > 0$ (negative control)

Table 6. Paired comparisons against the full method, pooled over heterogeneity levels and 10 seeds. Positive values favour the full method. p from a two-sided Wilcoxon signed-rank test on seed-paired differences.

Structure	Compared with	$\Delta C_{\max}$ (%)	n	p
[TBD: fill from clbs/experiments/gains.csv**]**				

we measure. Each cell is therefore run with 10 seeds, and every comparison is paired on the seed and tested with a two-sided Wilcoxon signed-rank test. Pairs are formed by explicit key—(instance, seed), or (H, seed) when comparing congestion structures—rather than by position in a list, a distinction that matters whenever a cell is incomplete.

Correctness. Every solution is re-checked by a validator that is independent of the decoder and re-derives corridor occupancies from the timetable, and is compared against the instance's composite lower bound. Runs that fail validation or fall below the bound are reported separately and never averaged into a cell. Each completed run is appended to a ledger and flushed, so an interrupted batch resumes without recomputation and reports are rebuilt from the ledger rather than from memory.

Experiments were run single-threaded on **[TBD:** CPU model**]**; the implementation is pure Python with no solver dependency.

5.3 Baselines and the ablation chain

The chain in Table 5 closes one link at a time, so that each step isolates one mechanism. Only the last entry is not part of the chain: it is a negative control, reported because it fails.

Row 1 is reported for scale but excluded from the budgeted comparisons: it performs a single decoding, so an equal-time budget has no meaning for it. Rows 4 and 5 are labelled by what they add; the difference between rows 5 and 6 isolates the staggering operator, and between rows 3 and 4 the decision path as a whole.

5.4 Main results

Fig. 1 shows the chain on both congestion structures, normalized by each instance's lower bound so that cells with different bounds can be shown on one axis. Table 6 gives the paired comparisons against the full method.

[TBD: narrative of the main result once the matrix run completes**]**

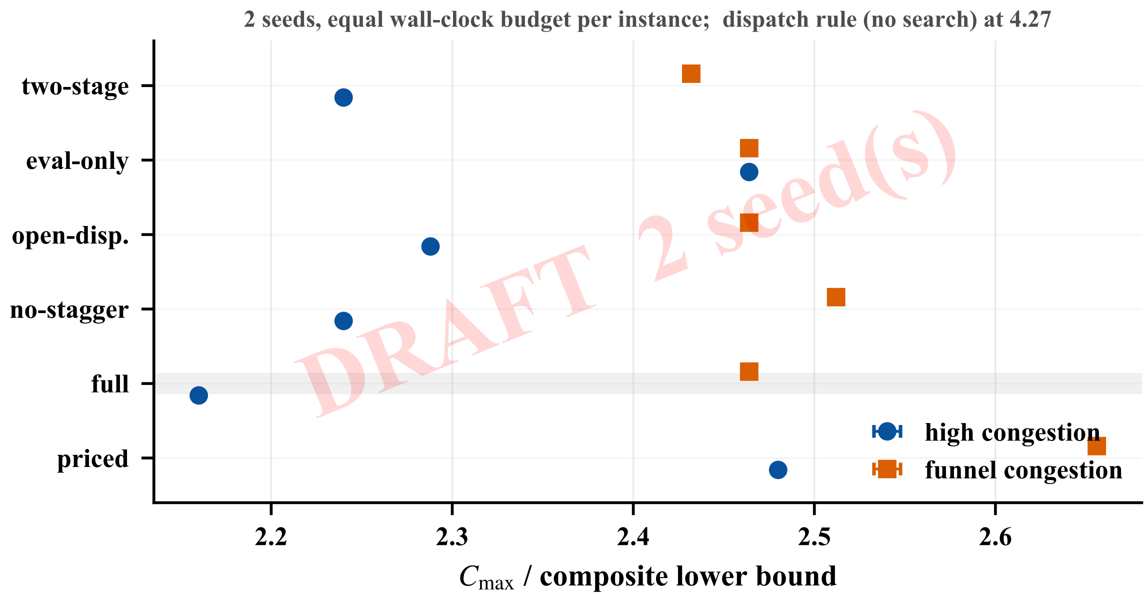

Fig. 1. The ablation chain under an equal wall-clock budget, as a ratio to the composite lower bound (lower is better). Error bars span heterogeneity levels. The chain separates on `high` congestion and collapses on `funnel`, where the added contention is unavoidable by construction.

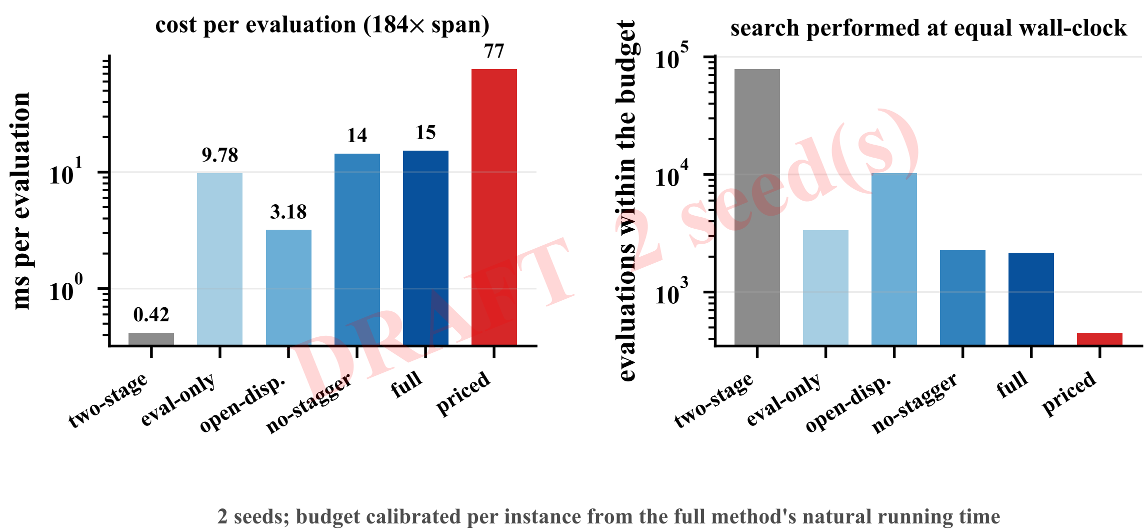

Fig. 2. Neither budget is neutral. Evaluation cost spans two orders of magnitude across the chain (left), so at equal wall-clock the cheaper arms complete far more search (right). Both quantities are reported for every cell.

Fig. 2 substantiates the protocol argument of Section 5.2 rather than the method: the left panel is why equal generations would be indefensible, and the right panel is why equal time is not a free choice either.

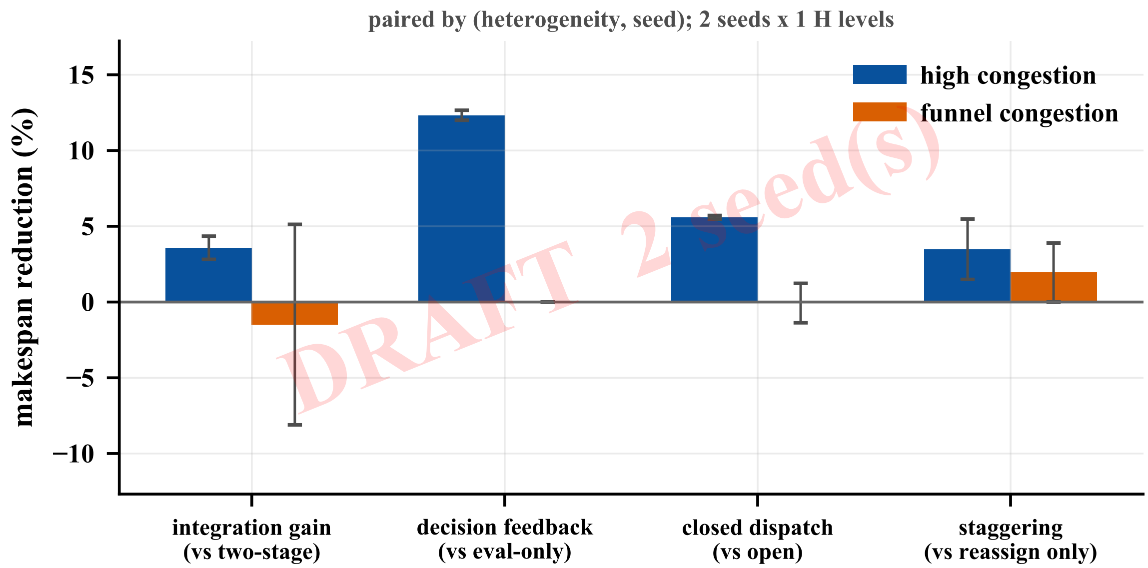

Fig. 3. Gains under avoidable (`high`) and unavoidable (`funnel`) congestion, paired by (H, seed). The two instance families differ only in the width of the station cut.

5.5 Which congestion is worth feeding back?

This is the sharpest test available to us, because `high` and `funnel` differ in one controlled respect. If the mechanisms work by steering decisions away from contention, their advantage must persist where contention is avoidable and vanish where it is not. A mechanism that improved both equally would be improving something else.

[TBD: result and interpretation**]**

The practical reading is a caveat on deployment rather than a claim of generality: the framework pays for itself where the layout leaves alternatives, and a shop whose traffic is dominated by a single mandatory bottleneck should widen the bottleneck instead. It also explains a discrepancy we would otherwise have had to leave unresolved. An earlier hand-built congested instance showed no advantage for the closed loop; measuring its structure afterwards revealed that its binding lower-bound term was the funnel bound, meaning a large share of its makespan was imposed by a corridor no decision could avoid. The mechanisms were being asked to act on a signal the instance did not contain.

5.6 Sensitivity to heterogeneity

Reassignment trades processing time for proximity, so it needs the two to differ. At $H = 0$ all eligible arms take the same time and the trade-off is empty; the operator can still move an operation to a less congested arm, but it buys nothing in duration and the resulting choice is arbitrary.

[TBD: result: does the gain grow with H on high, and stay flat on funnel?**]**

5.7 Pricing the interface: a negative result

Section 4.4 argued that pricing corridor occupancy double-counts a delay the fitness already sees. We test it directly by sweeping the coordination strength θ under the same wall-clock budget as every other configuration.

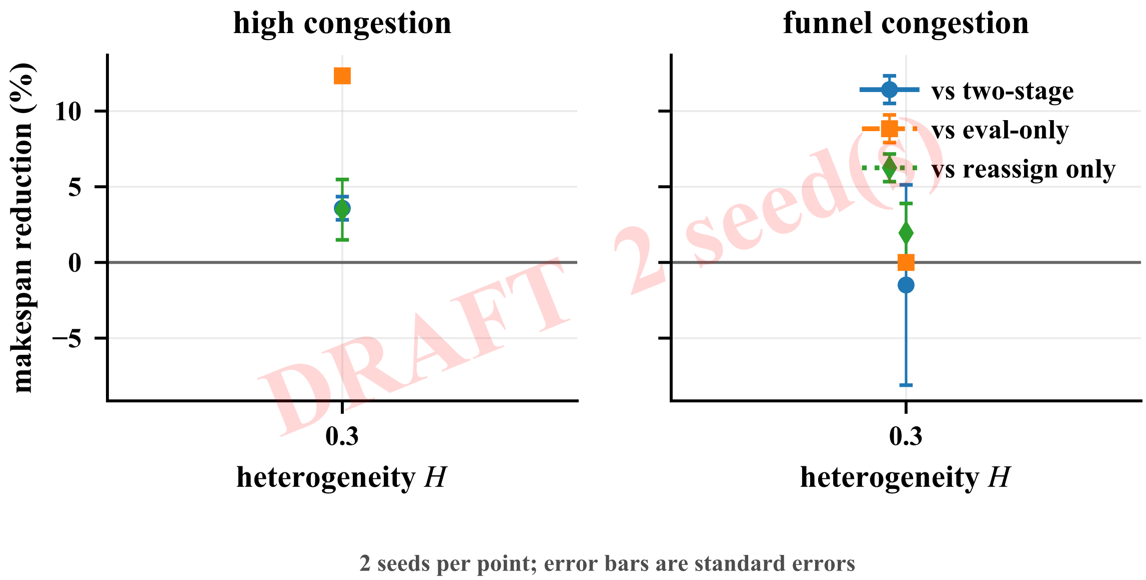

Fig. 4. Gain against heterogeneity. The mechanisms depend on arms differing in speed, which is what makes “reassign to a slower but better-connected arm” a decision rather than a coin flip.

[TBD: result: magnitude and monotonicity in theta]

We report this at length because the negative result is more transferable than a positive one would be. The argument does not depend on our price estimator: whether prices come from a finite-difference probe of corridor capacity or from a critical-chain surrogate, they charge for an occupancy whose consequence has already been paid, in full, inside the arrival time of the vehicle that yielded. Any architecture that evaluates fitness by a complete conflict-free decoding will have this property. The corollary is the design rule the paper adopts: upgrade the interface by making it carry structure, not price.

5.8 Threats to validity

Instance family. All cells share one size and one layout template. The factors we vary are the ones our analysis identifies as decisive, but size and topology are held fixed, and we make no claim about how the mechanisms scale.

Absence of a public benchmark comparison. The standard FJSP_PT benchmarks assume a constant travel matrix and provide no aisle network, so they cannot express vehicle contention; running on them would require deleting the constraint the paper is about. A degenerate comparison—our upper layer against published results with routing switched off—would test the genetic algorithm rather than the framework, and we have not performed it.

Bound quality. The composite lower bound is loose, with gaps to the best known solutions in the range of **[TBD:** range]%. It serves to keep the numbers honest and to catch regressions, not to certify optimality. Exact solutions for small instances, which logic-based Benders decomposition has been shown to deliver for this problem class [9], would settle how much of the remaining gap is search failure rather than model structure; we regard this as the most valuable next step.

Single implementation. Every arm shares one codebase, which removes implementation variance between arms but means the absolute makespans reflect the quality of that implementation. The ablation differences, being internal, are unaffected; comparisons against published methods would not be.

6 Conclusion

We addressed the CDHAS scheduling problem in intelligent manufacturing, where bidirectional spatiotemporal coupling between AGV trajectories and robotic arm task assignments creates a circular dependency that traditional sequential methods cannot resolve.

6.1 Summary of Contributions

We proposed the Hybrid Dynamic Programming (HDP) framework via a decompose-coordinate-converge strategy. Theoretically, we proved that the NP-hard coupled problem decomposes into tractable CSS-DP and DES-DP subproblems under mild separability conditions, with geometric convergence guarantees ($\rho = L_\Psi \cdot L_\Phi \cdot \alpha_{\text{crit}} < 1$) and polynomial task scalability $O(M^{1.8})$ (Sections 3–4). Practically, three acceleration techniques—reachability-based pruning, adaptive discretization, and warm-start initialization—reduce CSS state space by 60–80%, enabling HDP to scale to problem sizes where MILP times out. Empirically, HDP achieves near-optimal solution quality and significant makespan reduction with robust performance under uncertainty, confirming that coupling-aware joint optimization outperforms sequential baselines (Section 5).

6.2 Limitations and Future Directions

(1) Grid resolution sensitivity. Grid-based CSS-DP exhibits quadratic computational sensitivity $O((W/\Delta x)^2)$ and approximately 3% discretization error at $\Delta x = 0.5\,\text{m}$, which becomes prohibitive for high-precision applications ($\epsilon < 0.1\,\text{m}$). Future work will pursue adaptive discretization (dynamically adjusting grid resolution based on local obstacle density) and hybrid planning (combining grid-based DP for global routing).

(2) Agent scalability and TAMP integration. Scaling with CSS agents is constrained by quadratic collision avoidance complexity $O(N^2)$. Two directions are planned: decentralized coordination via consensus ADMM, where each CSS agent optimizes its own trajectory while exchanging only local coupling variables, and hierarchical decomposition that partitions large fleets into smaller teams coordinated by a higher-level scheduler. More broadly, embedding CSS-DP as a numeric trajectory optimization module within TAMP frameworks (e.g., PDDLStream) is a promising direction, where HDP's optimality guarantees and bidirectional coupling resolution complement symbolic high-level reasoning.

(3) Static environment assumptions. The current formulation assumes static obstacles and known task durations. Although HDP maintains performance under uncertainty (up to 40%, Section **??**), fully dynamic environments with online task arrivals and moving obstacles remain an open challenge. Rolling horizon optimization with warm-start re-planning provides a near-term direction, while robust DP formulations and predictive obstacle avoidance offer a longer-term direction.

References

[1] Giuseppe Fragapane, René de Koster, Fabio Sgarbossa, and Jan Ola Strandhagen. 2021. Planning and Control of Autonomous Mobile Robots for Intralogistics: Literature Review and Research Agenda. *European Journal of Operational Research* 294, 2 (2021), 405–426. doi:10.1016/j.ejor.2021.01.019

[2] Jie Fu, Bin Yang, Zhixing Chang, Yuanrong Zhang, Jiarui Wang, Xiaotong Wang, and Lei Wang. 2025. Integrated Production–Logistics Scheduling in Flexible Assembly Shops Using an Improved Genetic Algorithm. *Machines* 13, 12 (2025), 1090.

[3] M. R. Garey, D. S. Johnson, and Ravi Sethi. 1976. The Complexity of Flowshop and Jobshop Scheduling. *Mathematics of Operations Research* 1, 2 (1976), 117–129. doi:10.1287/moor.1.2.117

[4] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139

[5] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543

[6] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355

[7] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352

[8] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526

[9] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342

[10] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795

[11] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*.

Received 27 November 2025; revised 26 February 2026; accepted XX XX 2026